

ECE 647: Lecture 25

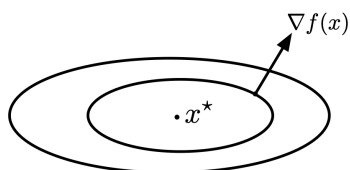
Scaled Gradient Descent Algorithm

Standard Gradient Algorithm

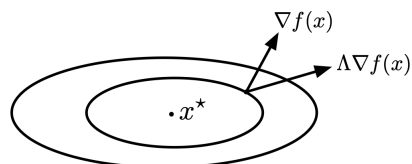
Update the parameter by:

$$x(t+1) = x(t) - \gamma \nabla f(x).$$

If $\nabla f(x)$ is Lipschitz and $\gamma < \frac{2}{L}$, then the algorithm converges.



(a) Standard gradient



(b) Scaled gradient

Scaled Gradient Algorithm

Update the parameter by:

$$x(t+1) = x(t) - \gamma \Lambda \nabla f(x),$$

where Λ is a positive-definite matrix. Some choice of Λ can make the algorithm converge faster.

- For example,

$$\Lambda = \begin{bmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_m \end{bmatrix},$$

each processor chooses different step-size.

- Regardless the use of Λ , the scaled gradient algorithm will converge under a similar Lipschitz condition:

$$\gamma < \frac{2}{L\lambda_{\max}},$$

where $\lambda_{\max}$ is the largest eigenvalue of Λ , i.e., $x^\top \Lambda x \leq \lambda_{\max} \|x\|^2, \forall x$.

Sketch of proof:

Choose a different norm:

$$\begin{aligned} & (x(t+1) - x^*)\Lambda^{-1}(x(t+1) - x^*) \\ &= (x(t) - x^*)\Lambda^{-1}(x(t) - x^*) + 2(x(t+1) - x(t))\Lambda^{-1}(x(t) - x^*) \\ & \quad (x(t+1) - x(t))\Lambda^{-1}(x(t+1) - x(t)) \end{aligned}$$

Note that

$$\begin{aligned} & (x(t+1) - x(t))\Lambda^{-1}(x(t) - x^*) = -\gamma \nabla f(x(t))(x(t) - x^*) \\ &= -\gamma [\nabla f(x(t)) - \nabla f(x^*)][x(t) - x^*] \leq -\frac{\gamma}{L} \|\nabla f(x(t))\|^2. \end{aligned}$$

Constraint Optimization

Consider the problem:

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & x \in X. \end{aligned}$$

The optimality condition is

$$f'(x; y - x) \geq 0, \forall y \in X.$$

If f is differentiable, then,

$$(\nabla f(x))^\top (y - x) \geq 0, \forall y \in X.$$

However, the normal gradient projection does not work any more because the new $x(t+1)$ can go outside X .

There are three solutions to address this issue:

1. *Penalty function method:* Choose $g(x)$ such that

$$\begin{cases} g(x) = 0, & \text{if } x \in X, \\ g(x) \geq 0, & \text{if } x \notin X. \end{cases}$$

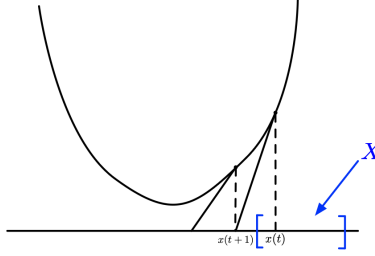


Figure 2: Going outside X .

- We can now minimize $f(x) + \beta g(x)$, and denote its solution as $x^*(\beta)$.
- As $\beta \rightarrow +\infty$, the penalty becomes larger and larger, the solution $x^*(\beta)$ will approach x^* (the original constrained problem).
- We will discuss the engineering implication of this approach when we discuss TCP as an example.

2. *Interior point method (Barrier method):*

- Choose $g(x)$ such that $g(x) \rightarrow +\infty$ as x approaches the boundary of X from inside.

For example:

$$\begin{aligned} X = \{x \geq 0\}, \quad g(x) &= -\log x. \\ X = \{x \leq a\}, \quad g(x) &= -\log(a - x). \end{aligned}$$

- We then minimize $f(x) + \beta g(x)$.
- Due to the barrier $g(x)$, the optimal solution $x^*(\beta)$ must be in the interior of X .
- As $\beta \rightarrow 0$, $x^*(\beta) \rightarrow 0$ (the original constrained problem).

In both the penalty-function method and interior-point method, the problem is converted to an unconstrained problem. Hence, we can use gradient algorithm to solve $x^*(\beta)$.

However, it may be difficult to ensure the Lipschitz condition of the gradient.